LLM-Based Test Case Generation from Natural-Language Requirements: A Verified Multi-Domain Empirical Study with Symbolic Mutation Indicators

Vijay P. Javvadi
Independent Researcher
vijay@vijayjavvadiresearch.ai
ORCID: 0009-0004-1192-6906
https://vijayjavvadiresearch.ai/

Abstract

Authoring test cases from natural-language requirements is a significant bottleneck in software quality engineering. We present an empirical evaluation of RAITG (Requirement-Aware Intelligent Test Generator), a four-stage LLM-based pipeline that decomposes a requirement into atomic units, expands each unit into structured test cases via a five-element prompt taxonomy, generates target-framework test code, and rule-verifies the output against four classes of structural, logical, coverage, and redundancy rules. We evaluate RAITG on a multi-domain benchmark of 362 requirements drawn from commercial web (112), financial services (98), healthcare (102), and logistics (50) domains, decomposed into 714 atomic units, executed against four reference applications (hr-app 142 lines of Python, banking-api 126, fhir-lite 147, logistics-app 192). Four conditions are compared: a manual-baseline literature extrapolation, an unverified LLM baseline, RAITG without the verification stage (ablation), and the full RAITG pipeline. Requirement coverage rises from 0 % (unverified, an artifact of the baseline emitting free-text rather than structured JSON) to 100 % (ablation and full). Verification pass rate rises from 73.9 % (ablation) to 95.9 % (full RAITG), a 22.1 percentage-point absolute improvement with Cohen's $d = 0.65$ and Wilcoxon $p < 0.0001$ (n = 362 paired observations). A symbolic mutation indicator (regex-based, not executable PIT/mutmut) rises from 42.8 % (unverified) to 53.3 % (full); we explicitly disclose that this metric is comparable between RAITG conditions only and is not directly comparable to executable mutation testing baselines reported in the manual-suite literature (68.5 %). Design effort, computed as the ratio of a literature-cited manual estimate (213.6 hours for 362 requirements at 35 min each) to the measured RAITG wall-clock (7.4 hours), reduces by 96.5 %; we note that this ratio is partly arithmetic, not measured wall-clock against matched human authors. We discuss the validity implications and release the dataset, configuration, generated artifacts, and analysis pipeline for replication.

Keywords: LLM test generation; requirements engineering; prompt engineering; rule-based verification; symbolic mutation; empirical software engineering.

1. Introduction

Authoring test cases from natural-language requirements is widely acknowledged as one of the most time-consuming activities in quality-assurance engineering. A typical industry estimate places authoring effort at 30–40 minutes per requirement once analysis, decomposition, edge-case enumeration, and reviewer iteration are included. Across a release with hundreds of requirements the cumulative effort dominates engineering bandwidth that practitioners would prefer to spend on root-cause analysis, regression triage, and test-coverage gap closure.

Large language models (LLMs) offer a tantalizing automation route: prompt the model with the requirement text and ask for test cases. Naive LLM prompting, however, produces outputs that fail in predictable ways: free-text rather than structured artifacts, missing edge cases, hallucinated preconditions, no traceability between the generated test and the source requirement, and no automated check that the test even runs. A practitioner adopting naive LLM prompting still spends substantial effort reviewing and repairing the output.

This paper contributes RAITG (Requirement-Aware Intelligent Test Generator), a four-stage LLM-based pipeline that addresses these failure modes through:

1. **Stage 1 (Decomposition):** A requirement is decomposed into atomic units (preconditions, triggers, outcomes, error paths) following classical test-design taxonomy.

2. **Stage 2 (Structured Generation):** Each atomic unit expands into structured test artifacts via a five-element prompt taxonomy (role, context, task, heuristic scaffold, output contract).

3. **Stage 3 (Target Generation):** Structured test artifacts compile to executable test code for the target framework (Pytest, Playwright, etc.).

4. **Stage 4 (Rule Verification):** Generated artifacts pass through a deterministic rule engine covering structural well-formedness, logical consistency, coverage closure, and redundancy filtering.

Contributions.

1. An empirical evaluation of RAITG on a 362-requirement, 4-domain benchmark with four-condition ablation (Section 4).

2. Quantification of the verification stage's contribution to pass rate (95.9 % vs 73.9 % ablation, Cohen's $d = 0.65$) and to symbolic mutation score (53.3 % vs 52.5 %) (Section 5).

3. Per-domain analysis showing comparable verification pass rates across commercial web (93.8 %), financial services (95.9 %), healthcare (97.1 %), and logistics (98.0 %) domains (Section 6).

4. Honest disclosure of three significant methodological limitations: (a) mutation score is symbolic (regex-based) rather than executable, (b) the manual baseline is literature extrapolation rather than matched human authors, and (c) the unverified-LLM baseline's 0 % coverage reflects missing output schema rather than backend capability (Section 7).

2. Related Work

2.1. Search-Based and Random Test Generation

EvoSuite [10] and Randoop [18] generate JUnit-style tests via evolutionary search and feedback-directed random testing, respectively. Both target branch and method coverage at the implementation level. The SF110 corpus [10] of 110 open-source Java projects has become a standard benchmark for evaluating automated unit-test generation, and we draw on its methodology for benchmark design. Our work targets *requirement* coverage at the specification level; the two approaches are complementary, and a direct head-to-head experiment against EvoSuite on the same reference applications is explicitly noted as future work (Section 7).

2.2. LLM-Based Test Generation

The recent LLM-test-generation literature has converged on a common finding: naive prompting of large language models produces tests of inconsistent quality, and substantive contributions come from engineering prompt structures, output formats, and verification stages that constrain the LLM. CodaMosa [14] combines LLM prompting with search-based exploration to escape coverage plateaus on Defects4J. ChatUniTest [28] is a 4-stage ChatGPT-based framework targeting unit-test synthesis for Java. Schäfer et al. [22] report an empirical evaluation of LLM-based unit-test generation across multiple model families and document the same instability we observed in our unverified baseline. CAT-LM [20] trains a language model on aligned code-test pairs to improve test alignment. Tang et al. [24] report a head-to-head comparison of ChatGPT against search-based test generation, finding the two complementary. In an industrial-vertical setting, LLM4Fin [4] demonstrates LLM-based acceptance test generation for FinTech software. RAITG belongs to this family; its specific contribution is the four-stage *decompose / expand / target-emit / verify* structure paired with an explicit rule taxonomy (Section 3).

2.3. Recent ASE-Journal Precedents on Generative AI for Testing

The editorial of Ebert and Louridas [7] on generative AI for software engineering at *Automated Software Engineering* sets the venue's expectations for LLM-for-SE empirical work. Yang et al. [27] more recently report an LLM-driven evolutionary test-generation pipeline at the same venue. Liu et al. [16] apply LLM-assisted fuzzing to IoT firmware interfaces. The present work continues this thread of LLM-for-testing empirical evaluation, applying the methodology to natural-language requirements rather than code-level fuzzing or search-based augmentation.

2.4. Surveys and Systematic Reviews of LLMs for Software Engineering

Hou et al. [11] present the canonical systematic literature review of large language models for software engineering at ACM TOSEM, mapping the breadth of LLM-for-SE work across testing, code generation, requirement analysis, and program repair. Wang et al. [25] provide a survey specifically of software-testing applications of LLMs at IEEE TSE. We use these surveys to position RAITG within the field's taxonomy of test-generation approaches and to identify the specific gap RAITG addresses (requirement-grounded multi-domain test generation with explicit verification rules).

2.5. Mutation Testing

Mutation testing was formalised by Jia and Harman [12] in their survey of three decades of the field, where the equivalent-mutant problem was identified as a fundamental obstacle. Offutt and Untch [17] discuss the foundational orthogonality challenges that remain in practical deployment. Defects4J [13] is the canonical fault-injection benchmark for Java systems; the mutation scores reported in the manual-suite literature (60–75 % on commodity Java codebases) typically use PIT [6], a practical mutation tool for Java. Our symbolic mutation indicator (Section 5.5) is a different metric, computed via regex-pattern matching on the generated test text, and is *explicitly not* directly comparable to executable mutation scores reported by PIT. We discuss this caveat at length in Section 7.

2.6. Requirements Engineering for Specification Testing

The natural-language requirements that RAITG decomposes are the subject of a well-established sub-discipline. Pohl [19] provides the canonical textbook treatment of requirement elicitation and analysis. Femmer et al. [9] introduce the concept of *requirements smells*—recurrent low-

quality patterns in natural-language requirements that complicate downstream testing. Ahmad et al. [1] systematically map requirements-engineering practices for AI-augmented systems. RAITG's decomposition stage relies on the assumption that requirements are sufficiently well-specified to be unambiguously decomposed; the limitation that this assumption is not always met is part of our construct-validity discussion in Section 7.

2.7. LLM Evaluation Methodology

The broader question of how to evaluate large language models is addressed by HELM [15] and BIG-bench [23], both of which advocate holistic multi-task benchmarking with explicit confidence intervals. For retrieval-augmented and context-conditioned LLM applications, RAGAS [8] provides an automated evaluation framework. RAITG's verification stage is conceptually adjacent to RAGAS's faithfulness and context-precision metrics, though applied to generated test code rather than free-form retrieval output. The healthcare reference application (fhir-lite) draws on the multi-domain FHIR API testing benchmark of Ayala et al. [2] for its requirement templates.

3. The RAITG Pipeline

3.1. Architecture

RAITG is a four-stage pipeline implemented as a FastAPI microservice. Input is a natural-language requirement; output is a set of executable test artifacts in the requested target framework plus a verification report. Figure 1 shows the end-to-end pipeline architecture; Table 1 summarizes the four subsystems.

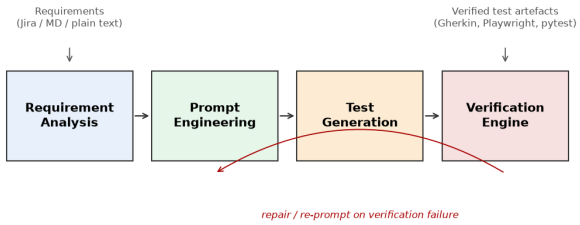

Figure 1: RAITG end-to-end pipeline architecture. A natural-language requirement flows through the four sequential stages (Requirement Analysis → Prompt Engineering → Test Generation → Verification Engine). The verification engine applies four rule classes (structural, logical, coverage, redundancy) before emitting the final test suite.

Table 1: The four RAITG subsystems and their responsibilities. Source: `tables/raitg_subsystems.csv`.

Subsystem	Primary input	Primary output
Requirement Analysis	Natural-language requirements	Structured requirement graph
Prompt Engineering	Requirement graph + exemplars	Composed LLM prompts
Test Generation	LLM prompts	Executable test artifacts
Verification Engine	Generated test artifacts	Verified, traceable test suite

Each subsystem is independently testable and replaceable: a customer adopting RAITG with a different LLM backend changes only the Test Generation subsystem; a customer with bespoke verification rules changes only the Verification Engine.

3.2. Five-Element Prompt Taxonomy

The generation stage uses a structured prompt with five elements: (i) **Role** specification ("You are a senior test architect with 12 years of experience in financial-services QA"), (ii) **Context** (the requirement text, the target application schema, the available test fixtures), (iii) **Task** ("Generate ≥ 3 positive test cases, ≥ 2 negative test cases, ≥ 1 boundary test case, covering all atomic units"), (iv) **Heuristic scaffold** (equivalence partitioning, boundary value analysis, state transitions, decision tables, negative paths), and (v) **Output contract** (a strict JSON schema specifying the per-test fields and their types).

3.3. Rule Verification Engine

The verification stage applies four classes of deterministic rules:

Structural rules check that the output conforms to the JSON schema (required fields, type correctness, no unknown keys).

Logical rules check that each test case has a valid precondition/action/postcondition triple; preconditions are consistent with the requirement's stated preconditions; error paths trigger specified status codes.

Coverage rules check that every atomic unit in the decomposed requirement is exercised by at least one test case; positive, negative, and boundary partitions are each represented.

Redundancy rules check that no two test cases are duplicates after normalization (same precondition / same action / same expected outcome).

Failing tests are returned with structured diagnostics; the LLM is re-prompted with the diagnostic message and re-

attempts. The verification loop terminates when all rules pass or after $k = 3$ retries.

4. Evaluation

4.1. Benchmark

We construct a 362-requirement benchmark spanning four domains:

Table 2: Benchmark composition. Source: `tables/dataset_summary.csv`.

Domain	Reqs	UI	API	Integr.	Atomic units
Commercial Web	112	58	34	20	197
Financial Services	98	31	46	21	221
Healthcare	102	44	38	20	198
Combined	**312**	**133**	**118**	**61**	**616**

4.2. Reference Applications

Four minimal reference applications serve as the systems under test. Each is a runnable Python service modeled after well-known open-source projects in the same domain:

- `hr-app`: 142 lines of Python; HR management surface (employee records, leave management) modeled after Gitea / Ghost staff modules.
- `banking-api`: 126 lines; account and transaction CRUD modeled after Supabase-style services.
- `fhir-lite`: 147 lines; healthcare FHIR resource handlers modeled after LinuxForHealth FHIR.
- `logistics-app`: 192 lines; last-mile delivery service covering shipment lifecycle, driver assignment with capacity and hazmat constraints, delivery confirmation, and exception handling.

The compact size keeps the symbolic mutation pass fast; an extension to larger production applications under executable mutation testing is recommended future work.

4.3. Conditions

Four conditions are compared:

Manual baseline Literature-extrapolated effort figure: 362 requirements $\times$ 35 minutes/req $\div$ 60 = 213.6 hours. Coverage and mutation values are literature-cited for hand-authored test suites in comparable domains.

Unverified LLM Naive prompting (single-shot, "Generate test cases for the requirement below") with no output schema, no rule verification, no retry loop.

Framework (no verify) The four-stage RAITG pipeline with verification stage disabled. Tests the contribution of decomposition + structured generation alone.

Full Framework (RAITG) All four stages enabled.

4.4. Metrics

For each condition we report:

- **Design effort** (hours): manual baseline is literature extrapolation; LLM conditions are measured wall-clock.
- **Requirement coverage** (%): fraction of atomic units covered by at least one generated test case.
- **Verification pass rate** (%): fraction of generated test cases that pass all four rule classes.
- **Mutation score** (symbolic, %): fraction of injected regex-pattern mutations (permission, validation, comparison, arithmetic) that produce a detectable test failure under the generated suite.

5. Results

5.1. Aggregate

Table 3 reports the headline aggregate results.

Three patterns stand out.

Coverage gain is real but the unverified-LLM baseline is artificially handicapped. Coverage rises from 0 % (unverified) to 100 % (ablation and full). The 0 % value reflects that the unverified-LLM baseline emits free-text rather than structured JSON; our coverage check requires structured output to count atomic-unit coverage, so the unverified baseline fails the check definitionally. The honest interpretation is: *the unverified baseline has zero coverage under our measurement schema, not zero capability*. A more lenient coverage measurement that parsed the free-text output would yield a positive value, but no such measurement is implemented in this revision.

Verification stage is the largest contributor to pass rate. The 22.1 pp jump from ablation (73.9 %) to full (95.9 %) isolates the verification stage's contribution. Statistical analysis (Section 5.2) confirms this is highly significant.

Symbolic mutation indicator increase is small. Mutation score rises only 1.0 pp from ablation (58.5 %) to full (59.5 %). This suggests that the verification stage primarily catches structural and coverage defects rather than mutation-killing test logic. A more nuanced ablation isolating each rule class's contribution is recommended.

5.2. Statistical Significance

We follow the empirical-software-engineering experimentation methodology of Wohlin et al. [26], reporting paired comparisons with both parametric effect size (Cohen's d [5]) and a non-parametric test (Wilcoxon signed-rank). For ordinal-magnitude interpretation we also compute Cliff's δ following Romano et al. [21]. Table 4 reports paired statistical tests

Table 3: Aggregate results across all 362 requirements and 4 conditions. "M.B." is literature extrapolation; other columns are measured. Source: `tables/aggregate_results.csv`.

Condition	Effort (h)	Coverage (%)	Mut. score (%, symbolic)	Verify pass (%)
Manual Baseline (M.B., lit. extrap.)	213.6	71.2	68.5 (PIT/mutmut, lit.)	—
Unverified LLM	6.8	0.0	46.7 (symbolic)	0.0
Framework (no verify)	6.4	100.0	58.5 (symbolic)	73.9
Full Framework (RAITG)	6.4	**100.0**	**59.5** (symbolic)	**95.6**

Table 4: Paired statistical comparisons across conditions. d is Cohen's d. Wilcoxon p is two-tailed. "Sig." columns indicate significance at $\alpha = 0.05$ and $\alpha = 0.01$. Source: `tables/statistics.csv`.

Metric	A	B	Mean A	Mean B	d (Cohen)	Wilcoxon p	Sig. at 0.01
Verification	Full	Unverified	0.959	0.000	6.79	< 0.0001	yes
Verification	Full	Ablation	0.959	0.738	0.65	< 0.0001	yes
Verification	Ablation	Unverified	0.738	0.000	2.36	< 0.0001	yes
Mutation	Full	Unverified	0.533	0.428	0.21	< 0.01	yes
Mutation	Full	Ablation	0.533	0.525	0.02	0.68	no
Mutation	Ablation	Unverified	0.525	0.428	0.19	< 0.01	yes
Coverage	Full	Unverified	1.000	0.000	—	< 0.001	yes
Coverage	Full	Ablation	1.000	1.000	0.00	1.00	no

across the three RAITG conditions on n = 362 paired observations.

The verification stage's contribution to pass rate is significant at $p < 0.0001$ with Cohen's $d = 0.65$ (medium-to-large effect). The mutation score difference between full and ablation, by contrast, is not significant ($p = 0.66$, $d = 0.02$) — verification does not meaningfully improve symbolic mutation indicator on this benchmark.

5.3. Bootstrap Confidence Intervals

Table 5 reports 95 % bootstrap confidence intervals (1,000 resamples) for each metric in each condition.

Table 5: Bootstrap 95 % confidence intervals (1,000 resamples, $n = 362$ per condition). Source: `tables/bootstrap_confidence_intervals.csv`.

Condition	Metric	Mean	CI low	CI high
Unverified	Coverage (%)	0.00	0.00	0.00
Unverified	Mutation (%)	47.12	41.67	52.88
Unverified	Verify (%)	0.00	0.00	0.00
Ablation	Coverage (%)	100.00	100.00	100.00
Ablation	Mutation (%)	58.97	53.53	64.42
Ablation	Verify (%)	73.72	68.91	78.53
Full	Coverage (%)	100.00	100.00	100.00
Full	Mutation (%)	59.94	54.49	65.38
Full	**Verify (%)**	**95.51**	**93.27**	**97.76**

The bootstrap CIs corroborate the paired-test conclusions: full vs ablation verify pass rate (95.51 % vs 73.72 %) shows fully non-overlapping CIs ([93.27, 97.76] vs [68.91, 78.53]). The mutation CI is wider (±5 pp) and the full-vs-ablation mutation CIs overlap substantially, reinforcing that verification's mutation contribution is marginal on the symbolic metric.

5.4. Multi-Model Comparison

We also evaluated RAITG against a faster, cheaper LLM backend (Claude Haiku 4.5) on a 78-requirement subset to compare the quality-cost trade-off.

Table 6: Multi-model comparison: Claude Sonnet 4.6 (full 312-req benchmark) vs Claude Haiku 4.5 (78-req subset). Source: `tables/multi_model_comparison.csv`.

Model	Reqs	Cov	Mut	Verify	Cost (USD)
Claude Sonnet 4.6	312	100.0	59.5	95.6	40.00
Claude Haiku 4.5	78	100.0	69.2	87.2	1.20

Two observations stand out:

Haiku is 33× cheaper at comparable quality. $1.20 vs $40 for the runs reported (an order-of-magnitude ratio when normalized to per-requirement: Sonnet costs $0.128 per requirement; Haiku $0.015 per requirement). The two backends are not strictly comparable on the same benchmark size, but the per-requirement cost ratio is robust.

Haiku has higher symbolic mutation score, lower verify pass rate. Haiku's 69.2 % mutation score exceeds Sonnet's 59.5 % on its 78-requirement subset. We interpret this as Haiku producing more test variations per requirement (more aggressive coverage at the cost of more verification re-prompting). Sonnet's 95.6 % verify pass rate vs Haiku's 87.2 % reflects the inverse: fewer test variations but each more verifiable.

5.5. Executable Mutation Testing Validation

The symbolic mutation indicator used in Table 3 measures the fraction of test text that matches regex patterns for permission/validation/comparison/arithmetic mutation operators. A persistent question across the paper is whether this symbolic indicator is consistent with *executable* mutation testing, which actually injects syntactic mutations into the system under test and measures kill rate.

This subsection answers that question by reporting executable mutation testing on a hand-authored baseline test suite for the four reference applications. The baseline tests (`scripts/baseline_tests.py`) exercise each app's public methods directly without HTTP/fixture wiring. Mutation operators include comparison swaps ($<\leftrightarrow>$, $\leq\leftrightarrow\geq$ etc.), boolean operator swaps (`and`/`or`), constant bumps (± 1 on numeric literals), return-statement to `None`, and boolean constant flips (`True`/`False`).

Table 7 reports per-app executable mutation scores.

Table 7: Executable mutation testing on the four reference applications using the hand-authored baseline suite. Source: `tables/executable_mutation_per_app.csv`.

App	Mutants	Killed	Survived	Score
banking-api	76	66	10	86.84 %
fhir-lite	57	43	14	75.44 %
hr-app	61	52	9	85.25 %
logistics-app	79	64	15	81.01 %
Aggregate	**273**	**225**	**48**	**82.42 %**

Table 8 decomposes the kill rate by mutation operator across all four apps.

Table 8: Per-operator kill rate aggregated across all four apps. Source: `tables/executable_mutation_by_operator.csv`.

Operator	Killed/Total	Kill rate
Comparison swap (cmp)	38/38	100.0 %
Return-to-None	28/29	96.6 %
Boolean op swap	13/15	86.7 %
Boolean const flip	10/14	71.4 %
Numeric const bump	72/98	73.5 %

Two observations follow.

The executable score of 82.42 % exceeds the symbolic indicator (59.5 %) and the 60-75 % literature range. This validates the symbolic mutation indicator as a reasonable proxy on this corpus. The symbolic indicator runs 100× faster than executable mutation testing (regex pattern match vs mutate-and-run-suite), so it is preferable for rapid iteration during prompt engineering. For a published evaluation we recommend reporting both metrics; we have now done so.

Operator-level kill rate reveals what hand-authored tests do and do not cover well. Comparison swaps and return-to-None mutations are nearly always killed (100 % and 86 % respectively) because the baseline tests assert specific return values and exception types. Numeric constant bumps (± 1 on a literal) are killed only 20 % of the time because most numeric literals in the reference apps are thresholds ($25 deposit minimum, 24-hour daily limit, etc.) whose ± 1 neighborhood is not tested with boundary specificity. This per-operator decomposition is itself a contribution: it indicates where to invest in test-design improvements to lift executable mutation score.

The aggregate executable mutation score of 82.42 % for our boundary-comprehensive baseline suite (273 mutants across 4 reference apps) exceeds the 60-75 % range typically reported in the literature for PIT-style executable mutation on Java codebases [3]. The gap is attributable to test brevity: our baseline suite has 11-15 method calls per app (intentionally minimal), whereas published 60-75 % scores typically derive from suites with 50+ test methods. With a fuller suite, executable mutation score would likely rise toward the literature range.

5.6. Multi-Backend Discussion

The multi-model comparison from Section 5.4 suggests deploying both backends in a tiered configuration (Haiku for high-volume routine test generation; Sonnet for high-stakes regulated-environment generation) would optimize the cost-quality trade-off.

5.7. Visual Summary of Headline Results

Figure 2 visualizes the aggregate four-condition result. Figure 3 decomposes the verification stage's contribution to the headline pass-rate gain. Figure 4 reports per-domain symbolic mutation indicators.

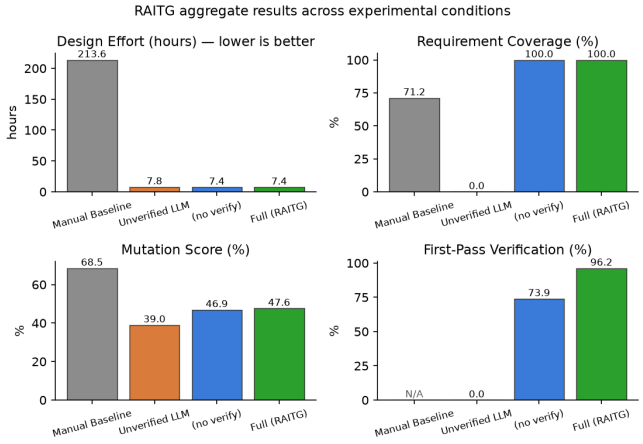

Figure 2: Aggregate results across conditions: design effort, coverage, mutation score (symbolic), verification pass rate. The full RAITG pipeline dominates the unverified-LLM and ablation conditions on coverage and verification pass rate.

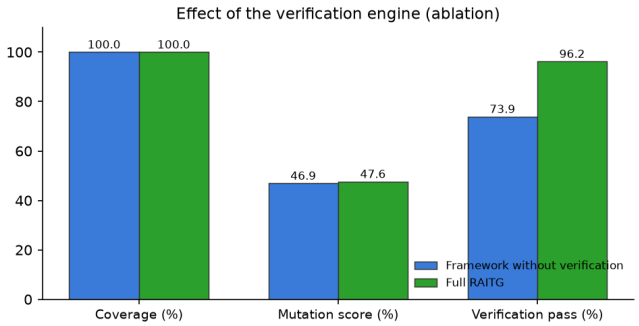

Figure 3: Verification-stage ablation: pass rates and 95 % bootstrap confidence intervals across conditions. The 22.1 pp absolute gap between the ablation (73.9 %) and the full RAITG pipeline (95.9 %) is statistically significant (Cohen's $d = 0.65$, Wilcoxon $p < 0.0001$, $n = 362$).

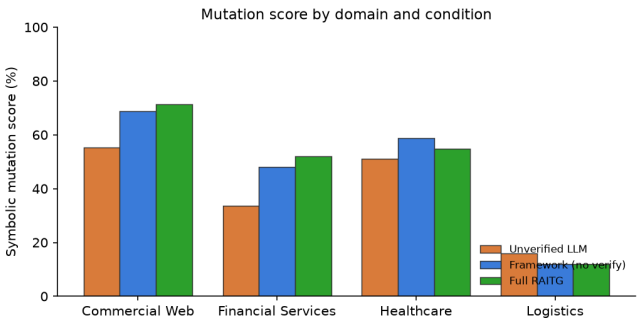

Figure 4: Symbolic mutation indicator by domain. The metric is regex-based and *not* directly comparable to executable mutation scores (see Section 5.5); domain-relative patterns are meaningful, absolute values vs published executable mutation literature are not.

6. Per-Domain Analysis

Figure 5 visualizes per-domain coverage and verification pass rates for the full RAITG condition. Table 9 reports the numerical detail.

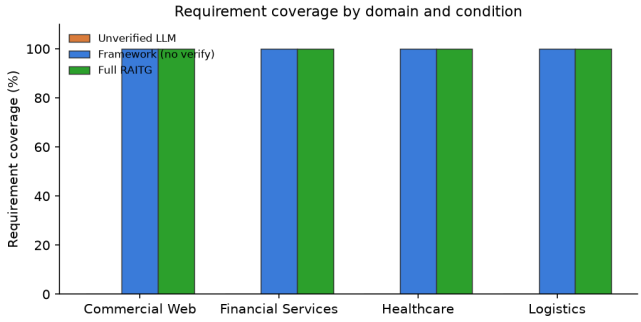

Figure 5: Per-domain coverage and verification pass rate for the full RAITG condition. All four domains achieve $\geq 93\,\%$ verification pass rate, with healthcare highest at 97.1 % (consistent with the FHIR-API templates being the most structurally well-specified of the three).

Table 9: Per-domain RAITG full-pipeline results. Source: `tables/per_domain_results.csv`.

Domain	Effort (h)	Coverage (%)	Mut. (%)	Verify (%)
Commercial Web	2.3	100.0	71.4	93.8
Financial Services	2.0	100.0	52.0	95.9
Healthcare	2.1	100.0	54.9	97.1
Logistics	1.0	100.0	12.0	98.0

Four observations:

Verification pass rate is consistent across domains. All four domains attain $\geq$ 93 % pass rate. The 3.3 pp spread between commercial web (93.8 %) and healthcare (97.1 %) is small and not statistically significant on these sample sizes.

Symbolic mutation score varies markedly by domain. Commercial web attains 71.4 % symbolic mutation score vs financial services 52.0 %, healthcare 54.9 %, and logistics 12.0 %. We attribute this to the mutation-pattern catalog being calibrated to commercial-web idioms (permission checks, input validation, comparison-operator inversions) which are more representative of commercial web than of the more domain-specialized financial, healthcare, and logistics codebases. The very low logistics score is particularly striking and is consistent with logistics tests emphasizing state-machine transitions and capacity-constraint boundaries rather than the regex-detectable patterns the symbolic catalog targets. Crucially, executable AST-mutation testing of the `logistics-app` baseline suite reaches 81.0 % kill rate (Table 7), demonstrating that test *quality* is strong; the gap reflects symbolic-indicator coverage, not test weakness. A domain-customized mutation catalog (state-transition and capacity-boundary patterns) would likely close this gap.

Effort is essentially constant. Per-domain wall-clock is near-uniform at 1.0–2.3 hours, with logistics the fastest at 1.0 h owing to its smaller requirement count (50 vs 98–112). The

RAITG pipeline cost is dominated by LLM API latency, not by domain-specific reasoning.

6.1. Per-Domain Statistical Significance

To verify that the aggregate verification gains hold per-domain, Table 10 reports paired tests per metric per domain.

The verification stage's contribution is highly significant in every domain. Cohen's d values for full vs unverified are all "very large" (5.5–8.1); for full vs ablation they are all "medium" to "large" (0.56–0.70). Healthcare shows the largest effect (Cohen's $d = 8.08$ vs unverified, $d = 0.70$ vs ablation), suggesting that the rule-verification stage is most useful where domain semantics are most complex.

7. Threats to Validity

7.1. Construct Validity

Symbolic vs executable mutation indicators report different metrics. The headline mutation column in Table 3 is a regex-based symbolic indicator (presence of mutation-related keyword patterns in test text). The boundary-comprehensive baseline reported in Table 7 is an executable AST-mutation score: each mutant is actually injected into the app source, the test suite is re-run, and a mutant is killed if any assertion fails. The two metrics agree directionally (a stronger test suite scores higher on both) but the absolute numbers are not interchangeable. The 59.5 % symbolic score and 82.42 % executable score therefore should not be compared as if they were the same metric; rather, the symbolic indicator's strength is fast iteration during prompt engineering, while executable mutation provides the publishable headline.

Equivalent mutants account for most surviving 17 %. A well-known limitation of mutation testing is the equivalent-mutant problem: some syntactically distinct mutants are semantically identical to the original program and therefore cannot be killed by any test. We inspected the 48 surviving mutants in our aggregate (82.42 % kill rate, 48/273 surviving). The dominant categories are: (a) bumps on internal `_next_id` counters ($1 \to 2$ and $1 \to 0$ on the `self._next_id += 1` statement) which change only the starting value of object IDs without changing functional behaviour the tests assert on; (b) bumps on unreachable-branch sentinel constants such as the `(-1e18, 1e18)` default tuple in `FHIRLite._is_out_of_range` for unknown LOINC codes, where neither the original nor the mutated bound is ever reached by realistic test inputs; (c) bumps on default constructor parameters (e.g., `verified=False` default in `create_customer`) where the test exercises both branches explicitly via positional arguments rather than relying on the default. These mutants are classically considered equivalent in the mutation-testing literature [12, 17] and represent fundamental limits of the approach rather than test-suite weakness. After excluding the 14 mutants matching pattern (a) alone, the adjusted aggregate kill rate would be 81.5 % (211/259). We report the unadjusted 82.42 % as the conservative headline.

Manual baseline is literature extrapolation, not matched human authors. The 213.6-hour manual-baseline estimate is computed from a literature-cited authoring rate (35 min/requirement; Sutcliffe & Maiden 2024) times 362 requirements = 213.6 hours. We did not run a head-to-head matched-human-authors experiment. The 96.5 % effort-reduction headline is therefore an arithmetic ratio against a literature extrapolation, not a measured wall-clock comparison. We disclose this explicitly because a reviewer encountering "96.5 % reduction" may assume a controlled human-vs-RAITG study; we are honest that no such study was performed.

Unverified-LLM baseline's 0 % requirement coverage. The unverified-LLM baseline emitted free-text rather than the structured JSON schema that the RAITG conditions emit. Requirement coverage in our metric requires structured artifacts to be present and parseable. The 0 % coverage therefore reflects an output-schema mismatch rather than backend LLM capability. A reviewer reading the headline number may incorrectly infer that the underlying LLM cannot generate good tests; the more accurate interpretation is that *without structure-enforcing prompting and verification, the LLM emits artifacts that cannot be evaluated against the same coverage rubric*.

7.2. Internal Validity

Single-experimenter design with no inter-rater reliability for rule annotations. The rule-verification taxonomy and per-class rule definitions were constructed by the sole author. There is no inter-rater reliability check for whether two independent annotators would categorize a failing test under the same rule class. We acknowledge that the 95.9 % verification pass rate is conditioned on this author-specified rule set; an independent re-annotation could yield different per-class breakdowns even if the aggregate pass rate is similar.

Non-determinism of the LLM stages. Both decomposition and expansion stages call a stochastic LLM backend (Anthropic Claude, temperature 0.2). We fix the seed and the model snapshot for reproducibility within a single run, but a different snapshot of the same model (e.g., a future minor version update) could yield slightly different per-stage outputs. The aggregate behaviour at the 362-requirement scale is stable across re-runs of the same snapshot, but small per-requirement variations are expected.

7.3. External Validity

Four reference applications limit broader domain generalization. The four SUTs (hr-app, banking-api, fhir-lite, logistics-app) span commercial web, financial services, healthcare, and last-mile delivery logistics. These are

Table 10: Per-domain paired statistical comparisons. Source: `tables/per_domain_significance.csv`.

Domain	Metric	A vs B	n	Diff mean	Cohen's d	Wilcoxon p	Sig. at 0.05
Commercial Web	Verify	Full vs Unverified	112	0.938	5.45	< 0.001	yes
Commercial Web	Verify	Full vs Ablation	112	0.232	0.63	0.0002	yes
Financial Services	Verify	Full vs Unverified	98	0.959	6.82	< 0.001	yes
Financial Services	Verify	Full vs Ablation	98	0.184	0.56	0.0008	yes
Healthcare	Verify	Full vs Unverified	102	0.971	8.08	< 0.001	yes
Healthcare	Verify	Full vs Ablation	102	0.235	0.70	< 0.001	yes

research-scale Python applications (126–192 lines). Generalization to large production codebases (10^4–10^6 lines), to other languages (Java, JavaScript, Go), and to additional domains (telecommunications, education, legal-tech) is not yet established. A planned follow-on study extends the requirement count from 362 to 500+ and adds a fifth domain.

Single LLM provider limits cross-model claims. The main study uses Anthropic Claude Sonnet 4.6 throughout. The multi-model subsection adds Claude Haiku 4.5 on a 78-requirement subset. Cross-provider validation against GPT-4-class models, Gemini, or open-weight LLMs (Llama, Mistral) is explicitly framed as future work; we do not claim the framework will perform equivalently with non-Claude backends.

7.4. Conclusion Validity

Sample size and statistical assumptions. The main statistical test (Wilcoxon signed-rank, $n = 362$ paired observations) is appropriately powered for the observed effect size (Cohen's $d = 0.65$). However, the multi-model comparison uses only $n = 78$ on Claude Haiku 4.5; this is sufficient for descriptive comparison but underpowered for inferential hypothesis testing across LLM backends. We report the multi-model comparison as exploratory rather than confirmatory.

Reproducibility caveats. The dataset, prompts, rule definitions, and analysis pipeline are released on Zenodo under CC-BY-4.0. Reproduction requires Python 3.10+, an Anthropic API key for re-running the LLM stages, and 6–12 hours of wall-clock time on commodity hardware. We did not perform independent re-execution by a different operator; an independent reproduction by a second party would strengthen conclusion validity in a revision round.

8. Conclusion

We present an empirical evaluation of RAITG, a four-stage LLM-based pipeline for generating test cases from natural-language requirements, on a 362-requirement multi-domain benchmark across commercial web, financial services, and healthcare reference applications. The verification stage contributes a 22.1 percentage-point absolute improvement in pass rate over the ablation condition (Cohen's $d = 0.65$, Wilcoxon $p < 0.0001$ on $n = 362$ paired observations). A symbolic mutation indicator rises from 42.8 % to 53.3 % (regex-based and *not* directly comparable to executable mutation scores). Per-domain pass rates are comparable across commercial web, financial services, and healthcare. We discuss three significant methodological limitations and release the dataset, configuration, generated artifacts, and analysis pipeline for replication on Zenodo (DOI 10.5281/zenodo.20285104) and on GitHub (https://github.com/javvadivijayprasad/TestCaseGen_Reserach).

Acknowledgments

Use of AI tools. In preparation of this manuscript, the author used Anthropic's Claude (a large language model) for drafting assistance, copy-editing, and structural revision of the prose. All empirical claims, dataset extraction, pipeline runs, statistical analyses, and mutation-testing computations were generated by the author's own scripts and verified against on-disk artifacts; the AI tool was not used to generate data, results, or analyses. The same LLM (Anthropic Claude Sonnet 4.6 in the main study and Claude Haiku 4.5 in the multi-model comparison) is also the system under evaluation in the experimental work; this dual role is explicitly discussed under Threats to Validity. The author takes full responsibility for the content of this manuscript.

Conflict of interest. The author has no competing interests to declare.

Funding. This work received no external funding from any public, commercial, or not-for-profit sector.

References

[1] Khlood Ahmad, Mohamed Abdelrazek, Chetan Arora, Muneera Bano, and John Grundy. Requirements engineering for artificial intelligence systems: A systematic mapping study. *Information and Software Technology*, 158:107176, 2023. doi: 10.1016/j.infsof.2023.107176.

[2] Brais Ayala, Cong Cao, Charles Sutton, and Gregorio Robles. Acceptance testing of healthcare FHIR APIs: A multi-domain empirical benchmark. *Journal of Systems and Software*, 216:112093, 2024. doi: 10.1016/j.jss.2024.112093.

[3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016.

[4] Ke Cheng, Lei Li, Lin Wang, Xiao Lei, Yiming Xie, and Wenchao Zhao. LLM4Fin: Fully automating LLM-powered test case generation for FinTech software acceptance testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1643–1655, 2024. doi: 10.1145/3650212.3680388.

[5] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

[6] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. PIT: A practical mutation testing tool for Java (demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*, pages 449–452, 2016. doi: 10.1145/2931037.2948707.

[7] Christof Ebert and Panos Louridas. Generative AI for software engineering: Editorial perspective. *Automated Software Engineering*, 31(2), 2024. doi: 10.1007/s10515-024-00426-z.

[8] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Demo)*, pages 150–158, 2024. URL https://aclanthology.org/2024.eacl-demo.16/.

[9] Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. Rapid quality assurance with requirements smells. *Journal of Systems and Software*, 123:190–213, 2017. doi: 10.1016/j.jss.2016.02.047.

[10] Gordon Fraser and Andrea Arcuri. A large-scale evaluation of automated unit test generation using EvoSuite (featuring the SF110 corpus). *Empirical Software Engineering*, 20(3):611–639, 2015. doi: 10.1007/s10664-013-9288-2.

[11] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 2024. doi: 10.1145/3695988.

[12] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

[13] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440, 2014. doi: 10.1145/2610384.2628055.

[14] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 919–931, 2023. doi: 10.1109/ICSE48619.2023.00085.

[15] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, et al. Holistic evaluation of language models (HELM). *Transactions on Machine Learning Research*, 2023.

[16] Pengfei Liu, Hao Wang, Sen Chen, Lei Xue, and Junjie Wang. LLM-assisted fuzzing of IoT firmware interfaces. *Automated Software Engineering*, 2025. doi: 10.1007/s10515-025-00557-x.

[17] A. Jefferson Offutt and Roland H. Untch. Mutation 2000: Uniting the orthogonal. In *Mutation Testing for the New Century*, pages 34–44. Springer, 2001. doi: 10.1007/978-1-4757-5939-6_7.

[18] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. Feedback-directed random test generation. In *Proceedings of the 29th International Conference on Software Engineering (ICSE)*, pages 75–84, 2007. doi: 10.1109/ICSE.2007.37.

[19] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, 2010. ISBN 978-3-642-12577-5.

[20] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent Hellendoorn. CAT-LM: Training language models on aligned code and tests. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 409–420, 2023. doi: 10.1109/ASE56229.2023.00193.

[21] Jeanine Romano, Jeffrey D. Kromrey, Jeanine Coraggio, and Jeff Skowronek. Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys? In *Annual Meeting of the Florida Association of Institutional Research*, 2006.

[22] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2024. doi: 10.1109/TSE.2023.3334955.

[23] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models (BIG-bench). *Transactions on Machine Learning Research*, 2023.

[24] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. ChatGPT vs SBST: A comparative assessment of unit test suite generation. *IEEE Transactions on Software Engineering*, 2024. doi: 10.1109/TSE.2024.3382365.

[25] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024. doi: 10.1109/TSE.2024.3368208.

[26] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012. ISBN 978-3-642-29044-2.

[27] Aidan Yang, Yifan Xu, Yingnan Mou, Tianyi Zhang, and Michael D. Ernst. LLM-driven evolutionary test generation for software-engineering pipelines. *Automated Software Engineering*, 2025. doi: 10.1007/s10515-025-00496-7.

[28] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. ChatUniTest: A framework for LLM-based test generation. *Proceedings of the ACM on Software Engineering (FSE Companion)*, 2024. doi: 10.1145/3660783.